



UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET



BACKUP I RESTORE MEHANIZMI KOD INFLUXDB BAZE PODATAKA

Modul: Softversko inženjerstvo

Student:
Anita Golubović, br. ind. 2095

Mentor:
dr Aleksandar Stanimirović

Niš, maj 2026. godina

Sadržaj

1.	Uvod.....	1
2.	Kontekst backup-a i restore-a u prethodnim verzijama	2
3.	Arhitektura InfluxDB 3 Core sistema	3
3.1.	FDAP stek	3
3.2.	Tok podataka pri upisu.....	5
3.2.1.	Komponente storage engine-a.....	5
3.2.2.	Faze procesa upisa	6
3.3.	Object storage	7
3.4.	Struktura podataka na disku.....	8
4.	Odsustvo ugrađenih alata.....	10
5.	Backup mehanizam.....	11
5.1.	Preporučeni redosled backup-a	11
5.2.	Kako izvoditi backup	11
5.3.	Backup na lokalom fajl sistemu	11
5.4.	Backup na S3-kompatibilnom skladištu	12
5.5.	Šta backup ne obuhvata	13
6.	Restore mehanizam.....	14
6.1.	Priroda restore operacije	14
6.2.	Preporučeni redosled restore-a.....	14
6.3.	Restore na lokalnom fajl sistemu.....	14
6.4.	Restore na S3-kompatibilnom skladištu	15
6.5.	Očekivanja nakon restore-a i ograničenja oporavka	16
6.6.	Posebna razmatranja za Docker okruženje	16
7.	Praktična primena	17
7.1.	Backup	18
7.1.1.	Priprema i verifikacija stanja pre backup-a	18
7.1.2.	Izvođenje backup procedure	18
7.1.3.	Verifikacija sadržaja backup-a	20
7.2.	Restore	21
7.2.1.	Simulacija gubitka podataka	21
7.2.2.	Izvođenje restore procedure.....	21
7.2.3.	Problem sa dozvolama i njegovo rešavanje	23
7.2.4.	Verifikacija uspešnosti restore-a	24

5.	Zaključak.....	26
6.	Literatura.....	27

1. Uvod

Vremenske serije predstavljaju jedan od najbrže rastućih oblika podataka u savremenom računarstvu. Svaki IoT uređaj, svaki monitoring sistem za servere i mreže i svako industrijsko postrojenje generišu nizove vrednosti označenih vremenskim oznakama. Specifičnost ovih podataka je da se zapisuju velikim protokom, retko modifikuju nakon upisa, a tipično čitaju u vremenskim opsezima i agregirano. Karakteristika vremenskih serija je da jednom izgubljeni podaci najčešće ne mogu biti rekonstruisani, na primer senzor koji nije beležio merenja u određenom periodu ne može naknadno popuniti tu prazninu. Zbog toga zaštita podataka kroz mehanizme backup-a i restore-a predstavlja kritičan aspekt administracije ovakvih sistema.

U radu pred Vama analiziraju se mehanizmi backup-a i restore-a u najnovijoj generaciji InfluxDB sistema za upravljanje bazama podataka. Ova generacija isporučuje se kao InfluxDB 3 Core, koja je besplatna i otvorena verzija namenjena individualnim korisnicima i manjim timovima i InfluxDB 3 Enterprise, namenjena velikim organizacijama sa naprednim zahtevima.

U prvom delu rada prikazani su kontekst i motivacija, odnosno kako su backup i restore bili realizovani u ranijim verzijama sistema (InfluxDB 1.x i 2.x) i koje su arhitektonske promene u novoj generaciji uslovile potpuno drugačiji pristup ovim procedurama.

Zatim, u centralnom teorijskom delu fokus je na arhitekturi perzistencije podataka u InfluxDB 3 Core i detaljnom opisu backup i restore procedura, kroz koje se pokazuje da ispravno izvođenje ovih operacija zahteva duboko razumevanje interne organizacije sistema.

Praktični deo rada zasniva se na IoT sistemu sa tri senzora raspoređena u tri različite prostorije, koji na svakih 15 sekundi generišu i šalju podatke o temperaturi i vlažnosti. Na osnovu prikupljenih podataka demonstrira se kompletna backup i restore procedura u Docker okruženju, uključujući verifikaciju uspešnosti oporavka i rešavanje problema sa kojima se administrator može susresti u praksi.

2. Kontekst backup-a i restore-a u prethodnim verzijama

U prethodnim generacijama InfluxDB sistema backup i restore bili su realizovani kroz namenske CLI komande: **influxd backup** u verziji 1.x i **influx backup** u verziji 2.x. Korisnik je imao poseban alat koji je sakrivao detalje interne organizacije fajlova i sprovodio celokupnu proceduru kroz jednu komandu.

U InfluxDB 3 Core ovaj pristup je napušten kao posledica potpuno nove arhitekture zasnovane na object storage-u i Apache Parquet formatu. Backup i restore više se ne posmatraju kao komande koje rade nad jednim zatvorenim formatom, već kao postupci kopiranja i vraćanja više međusobno zavisnih komponenti: Parquet fajlova, WAL segmenata, Catalog-a i snapshot fajlova. Upravo zbog toga je razumevanje interne arhitekture sistema neophodan preduslov za ispravno izvođenje ovih procedura, što je i tema narednog poglavlja.

3. Arhitektura InfluxDB 3 Core sistema

Razumevanje načina na koji InfluxDB 3 Core interno skladišti i organizuje podatke predstavlja neophodan preduslov za razmatranje backup i restore mehanizama. Za razliku od tradicionalnih relacionih sistema baza podataka koji podatke smeštaju u binarne fajlove na lokalnom disku pod sopstvenom kontrolom, InfluxDB 3 Core je od temelja projektovan oko koncepta object storage-a kao jedinog sloja trajne perzistencije. Ova arhitektonska odluka direktna je posledica prelaska na novi tehnološki temelj, takozvani FDAP stek, i ima dalekosežne posledice na to šta backup i restore znače u praksi, kao i koje komponente je neophodno obuhvatiti i kojim redosledom to učiniti.

3.1. FDAP stek

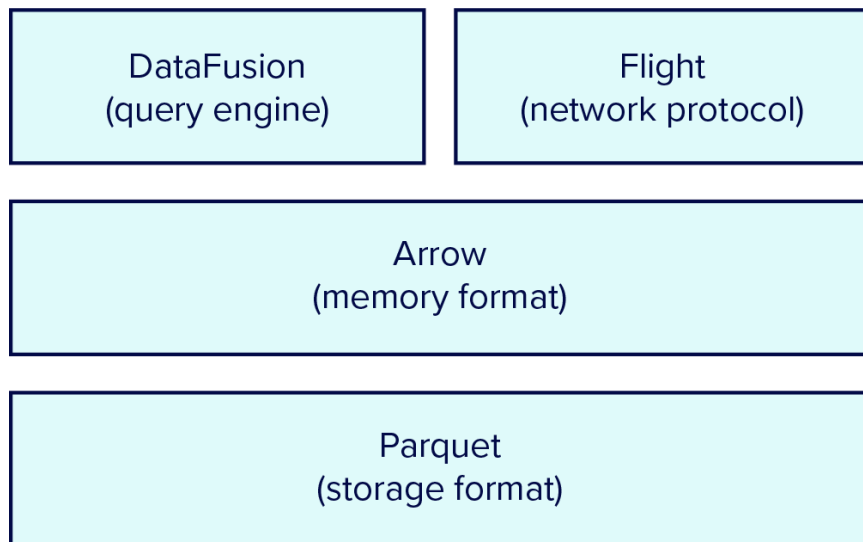
Skraćenica FDAP označava četiri Apache projekta. Komponente se ovde opisuju od sloja skladištenja ka sloju mrežne komunikacije, što odgovara putu kojim podaci prolaze od diska do klijenta.

Apache Parquet je kolumnarni format za trajno skladištenje podataka na disku. Kolumnarni format znači da se vrednosti iste kolone čuvaju uzastopno, što omogućava efikasnu kompresiju i selektivno čitanje samo onih kolona koje su relevantne za određeni upit. Parquet fajlovi interno su organizovani u logičke segmente zvane Row Groups, koji pored samih podataka sadrže i bogat skup metapodataka na nivou fajla i segmenta, odnosno statistike poput minimalnih i maksimalnih vrednosti po koloni. Upravo ovi metapodaci omogućavaju query engine-u da preskoči čitanje celih segmenata koji ne zadovoljavaju uslove upita, bez potrebe da ih fizički učita. Parquet fajlovi su centralni objekat koji backup mora da obuhvati, jer predstavljaju jedini trajni zapis sačuvanih podataka.

Apache Arrow je kolumnarni format za rad sa podacima u memoriji. Kada se Parquet fajlovi čitaju sa diska, podaci se dekodiraju direktno u Arrow strukture, čime se izbegavaju dodatne transformacije i kopiranja podataka u memoriji. Isti Arrow format koristi se i za čuvanje podataka u memorijskom baferu tokom upisa, što znači da podaci od trenutka prijema do trenutka trajnog zapisivanja nikada ne menjaju fundamentalnu strukturu.

Apache DataFusion je query engine implementiran u programskom jeziku Rust, koji koristi Arrow kao internu reprezentaciju podataka i omogućava izvršavanje SQL upita. Zahvaljujući poznavanju strukture i metapodataka Parquet fajlova, DataFusion može da primeni niz optimizacionih mehanizama koji smanjuju količinu podataka koja se fizički čita sa diska, što je posebno važno u scenarijima sa velikim brojem istorijskih fajlova.

Apache Arrow Flight je RPC protokol za prenos Arrow podataka između servera i klijenata. Rezultati upita prenose se direktno u Arrow formatu, bez potrebe za serijalizacijom u tekstualne formate poput JSON-a, čime se značajno smanjuje overhead pri prenosu velikih skupova podataka.



Slika 1 – FDAP stek

Row 1	XY-345 Thermometer California
Row 2	XY-799 Thermometer Delaware
Row 3	AA-609 Humidity California

Slika 2 – Primer row oriented formata

Sensor ID	Type	Location
XY-345	Thermometer	California
XY-799	Thermometer	Delaware
AA-609	Thermometer	California

Slika 3 – Primer column oriented formata

Kao posledica usvajanja FDAP pristupa, InfluxDB 3 Core više se ne posmatra kao zatvoren, specijalizovan sistem sa sopstvenim formatima podataka, već kao kombinacija standardnih, široko

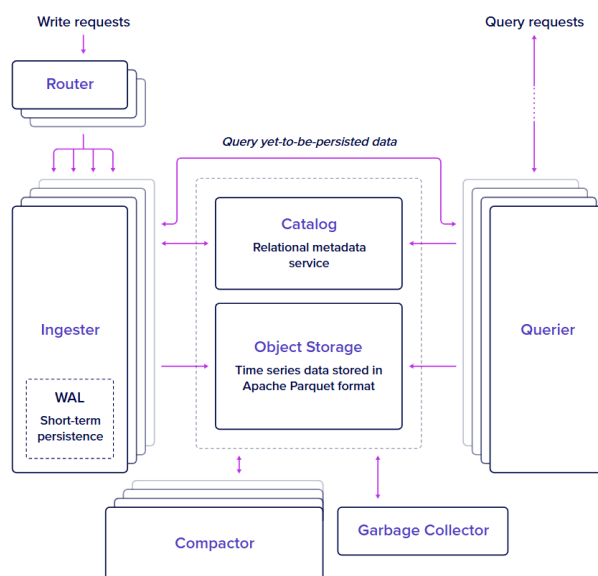
prihvaćenih tehnologija za skladištenje i obradu podataka. Za temu ovog rada ključna je činjenica da su Parquet fajlovi, kao standardni, otvoreni format, čitljivi i van samog InfluxDB sistema, što u određenim scenarijima otvara mogućnost direktnog pristupa podacima i bez procedure restore-a.

3.2. Tok podataka pri upisu

3.2.1. Komponente storage engine-a

Kako bi se razumeo tok upisa podataka, neophodno je najpre upoznati osnovne komponente storage engine-a:

- **Router** prima dolazne write zahteve i prosleđuje ih odgovarajućem Ingester-u na osnovu ciljne baze podataka.
- **Ingester** validira podatke, smešta ih u memorijski bafer u kolumnarnom formatu i zapisuje ih u WAL. Po potrebi ažurira šemu u Catalog-u.
- **WAL** (*Write-Ahead Log*) obezbeđuje trajnost podataka pre njihovog konvertovanja u Parquet format. U slučaju pada sistema, WAL omogućava oporavak podataka koji još uvek nisu sačuvani kao Parquet fajlovi.
- **Catalog** upravlja metapodacima, uključujući šemu, tabele i lokacije Parquet fajlova.
- **Object Storage** predstavlja trajno skladište u kome se podaci čuvaju u obliku Parquet fajlova, WAL segmenata i Catalog fajlova.
- **Compactor** je pozadinska komponenta koja periodično preuzima najstarije podatke iz memorijskog bafera, konvertuje ih u Parquet fajlove i upisuje ih u object storage. Pored toga, spaja manje Parquet fajlove u veće i optimizuje njihov raspored radi efikasnijeg čitanja.
- **Querier** izvršava korisničke upite, kombinujući podatke iz memorijskog bafera i Parquet fajlova.



Slika 4 – Komponente storage engine-a

3.2.2. Faze procesa upisa

Sa opisanim komponentama, proces upisa može se posmatrati kroz šest jasno definisanih faza:

1. **Write Request:**

Klijent šalje podatke u Line Protocol formatu putem HTTP API-ja (endpoint `/api/v3/write_lp`). Zahtev preuzima Router, koji ga prosleđuje Ingestor-u zaduženom za ciljnu bazu podataka.

Line Protocol je tekstuani format koji se koristi za unos podataka. Njegova osnovna sintaksa je:

measurement,tag_key=tag_value field_key=field_value timestamp

The diagram shows the Line Protocol syntax: `myTable,tag1=val1,tag2=val2 field1="v1",field2=1i 000000000000000000000000`. Above the string, four labels are positioned with vertical lines pointing to their respective parts: 'table' points to 'myTable', 'tag set' points to ',tag1=val1,tag2=val2', 'field set' points to 'field1="v1",field2=1i', and 'timestamp' points to '000000000000000000000000'.

Slika 5 – Elementi line protocol-a ("table" je sinonim za measurement)

2. **Validacija i Memory Buffer:**

Ingestor validira sintaksu i tipove podataka i, po potrebi, ažurira šemu u Catalog-u (npr. dodaje novu tag kolonu ili polje). Validirani podaci se zatim smeštaju u memorijski bafer u kolumnarnom formatu zasnovanom na Apache Arrow.

U ovoj fazi još uvek ne postoje trajne strukture na disku, ali kolumnarna organizacija podataka, gde se vrednosti iste kolone nalaze uzastopno u memoriji, predstavlja prvi korak ka efikasnom čitanju i filtriranju.

3. **WAL Persistencija:**

Sadržaj bafera se približno jednom u sekundi (kontrolisano parametrom **--wal-flush-interval**) zapisuje u Write-Ahead Log (WAL) na object storage-u. WAL obezbeđuje trajnost podataka i omogućava oporavak u slučaju pada sistema, sve dok podaci postoje u WAL-u, mogu biti rekonstruisani čak i ako Parquet fajlovi još uvek nisu kreirani.

4. **Queryable Buffer:**

Nakon WAL persistencije, podaci postaju dostupni za upite kroz queryable buffer. Sistem podrazumevano u ovom baferu drži do 900 WAL segmenata, što odgovara otprilike 15 minuta podataka. Ovim se omogućava brz pristup najnovijim podacima bez čitanja sa object storage-a, što je posebno važno u scenarijima sa visokom frekvencijom upisa kao što su IoT i monitoring sistemi.

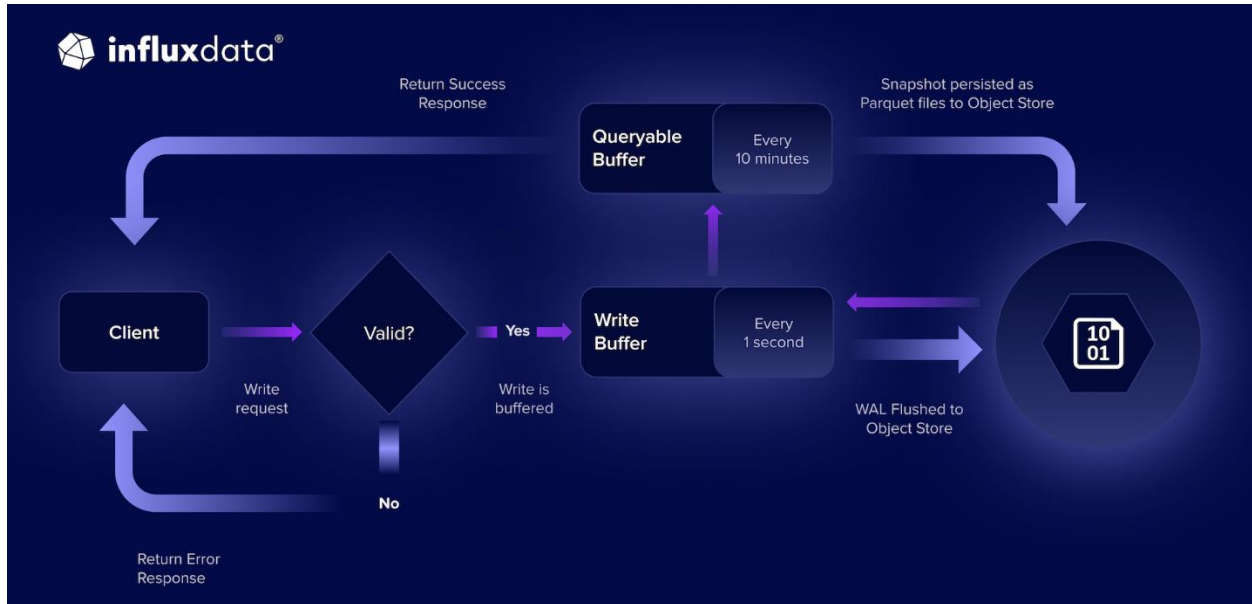
5. **Parquet Persistencija (snapshot):**

Na svakih deset minuta (podrazumevana vrednost), Compactor preuzima najstarije podatke iz queryable bafera i trajno ih zapisuje u Parquet fajlove na object storage-u. Lokacija svakog novokreiranog fajla registruje se u Catalog-u, a odgovarajući WAL segmenti koji su uspešno konvertovani mogu biti obrisani. Istovremeno se kreira snapshot fajl koji beleži koje Parquet fajlove sistem prepoznaje kao konzistentno stanje. Ovaj mehanizam je od

posebnog značaja za backup i restore procedure, jer upravo snapshot fajlovi definišu do koje tačke u vremenu je oporavak moguć.

6. In-Memory Parquet Cache:

Nedavno kreirani Parquet fajlovi se drže u memoriji Querier-a, što dodatno smanjuje potrebu za pristupom object storage-u pri upitima nad skorašnjim podacima.



Slika 6 – Tok upisa podataka

3.3. Object storage

InfluxDB 3 Core podržava više backend-ova za object storage, što sistemu daje značajnu fleksibilnost u pogledu okruženja u kojima može da se primeni.

Podržani backend-ovi su:

- lokalni fajl sistem (lokalni direktorijum),
- AWS S3 i S3-kompatibilna skladišta (kao što je MinIO),
- Azure Blob Storage i
- Google Cloud Storage.

Bez obzira na koji od navedenih backend-ova se koristi, logička organizacija podataka ostaje identična. Svi podaci i svi metapodaci instance smeštaju se unutar hijerarhije direktorijuma čiji je koren imenovan prema identifikatoru čvora (*node_id*). Ovaj identifikator je jedinstven za svaku instancu i konfiguriše se pri pokretanju servera. Sve datoteke koje čine stanje jedne instance nalaze se isključivo unutar te putanje, što backup proceduru čini logički dobro ograničenom jer nema rasutih sistemskih fajlova van ove hijerarhije koje bi trebalo posebno pratiti.

Važno je napomenuti da memory-based object store, koji InfluxDB 3 Core može da koristi u određenim scenarijima, nije podržan za backup i restore operacije. Ovaj tip skladišta ne obezbeđuje trajnost podataka između restartova sistema, pa samim tim i nije relevantan u kontekstu zaštite podataka.

3.4. Struktura podataka na disku

Sve datoteke koje čine stanje instance organizovane su unutar hijerarhije sa korenom **<node_id>/**. Svaki element ove strukture ima jasno definisanu ulogu i različitu važnost u kontekstu oporavka sistema.

Direktorijum **<node_id>/catalog/** sadrži log fajlove koji prate sve promene kataloškog stanja, uključujući kreiranje baza podataka, kreiranje tabela, izmene šeme i slične operacije nad metapodacima. Catalog je fundamentalna komponenta: bez njega sistem ne može da interpretira šta Parquet fajlovi sadrže niti kako su podaci organizovani.

Fajl **<node_id>/_catalog_checkpoint** predstavlja sažeti snimak trenutnog stanja Catalog-a. Umesto da pri svakom pokretanju čita čitav Catalog log od početka, sistem učitava ovaj checkpoint fajl i od njega nastavlja. To ga čini kritičnom tačkom za brz i ispravan oporavak.

Direktorijum **<node_id>/wal/** sadrži Write-Ahead Log fajlove koji predstavljaju podatke koji su trajno prihvaćeni, ali još uvek nisu konvertovani u Parquet format. Backup WAL-a je neophodan za očuvanje podataka upisanih između poslednjeg Parquet ciklusa i trenutka kada se backup vrši.

Direktorijum **<node_id>/snapshots/** sadrži snapshot fajlove koji sumarizuju koji Parquet fajlovi postoje i koji je njihov sadržaj. Snapshot mehanizam omogućava sistemu da pri oporavku efikasno utvrdi konzistentno stanje, samo Parquet fajlovi na koje snapshot fajl eksplicitno referiše biće uzeti u obzir pri oporavku. Parquet fajlovi bez reference u snapshot-u se ignorišu.

Direktorijum **<node_id>/dbs/<baza>/<tabela>/<datum>/** sadrži same Parquet fajlove organizovane po bazi podataka, tabeli i vremenu. Ovo je ujedno i najveći deo ukupnog skladišnog prostora koji instanca zauzima, jer predstavlja kompletnu istoriju sačuvanih podataka.

Direktorijum **<node_id>/table-snapshots/<baza>/<tabela>/** sadrži tabele snapshot fajlove koji se periodično prepisuju i koji se automatski regenerišu pri svakom restartu servera. Iz tog razloga, ovi fajlovi se namerno isključuju iz backup procedure, njihovo odsustvo ne ugrožava ispravnost oporavka, a njihovo uključivanje samo povećava veličinu backup-a bez dodatne koristi.

Sledeća tabela daje pregled svih komponenti i njihove važnosti za backup i restore:

Putanja	Sadržaj	Važnost za backup i restore
<node_id>/_catalog_checkpoint	Checkpoint kataloga	Kritično – bez ovog fajla restore nje moguć
<node_id>/catalog/	Log promena kataloga	Kritično – definiše strukturu i šemu podataka
<node_id>/wal/	WAL fajlovi	Neophodno – sadrži podatke još uvek nepersistovane u Parquet formatu

<code><node_id>/snapshots/</code>	Snapshot fajlovi Parquet stanja	Neophodno – definiše do koje tačke u vremenu je oporavak moguć
<code><node_id>/dbs/</code>	Parquet fajlovi sa podacima	Neophodno – primarni sadržaj koji se čuva i vraća
<code><node_id>/table-snapshots/</code>	Tablea snapshot fajlovi	Izostaviti – regenerišu se automatski pri restartu

Za potpuno razumevanje stanja instance nisu dovoljni samo Parquet fajlovi sa vremensko-serijskim podacima. Catalog, WAL fajlovi i snapshot fajlovi jednako su neophodni.

Bez Catalog-a sistem ne može da interpretira šta Parquet fajlovi sadrže, bez WAL-a gube se podaci upisani nakon poslednjeg Parquet ciklusa, a bez snapshot fajlova sistem ne može da utvrdi koje Parquet fajlove treba da uzme u obzir pri oporavku.

Važno je uočiti da se sve čuva pod istim **node_id** direktorijumom i da svaki poddirektorijum ima jasnu, dokumentovanu ulogu. Ne postoje fajlovi izvan ove hijerarhije koje bi trebalo posebno tretirati, backup i restore procedure opisane u poglavlju 5 i 6 definisane su isključivo nad ovim skupom komponenti.

Zbog toga se backup ne sme svesti samo na kopiranje direktorijuma *dbs/*, a restore na njegovo vraćanje. Tek kada su sačuvane sve komponente: Catalog, WAL, snapshot-ovi i Parquet fajlovi, može se govoriti o konzistentnom backup-u iz kojeg je moguć potpun oporavak. Kopiranje komponenti u pogrešnom redosledu ili izostavljanje pojedinih komponenti rezultuje neusaglašenim snimkom, iz kojeg restore možda neće biti moguć ili će dovesti do nepotpunog stanja baze. Upravo zato restore nije trivijalna operacija kopiranja u suprotnom smeru, već zahteva pažljivo poštovanje propisanog redosleda.

4. Odsustvo ugrađenih alata

Jedna od prvih i najvažnijih karakteristika InfluxDB 3 Core jeste da sistem trenutno ne poseduje ugrađene alate za backup ni za restore. Ne postoji posebna komanda, API endpoint niti administrativni interfejs koji bi automatski kreirao konzistentnu kopiju podataka ili je vratio. Umesto toga, obe procedure zasnivaju se na ručnom kopiranju fajlova iz object storage-a, pri čemu odgovornost za ispravnost leži u potpunosti na administratoru sistema.

Ova odluka je direktna posledica arhitekture zasnovane na object storage-u, budući da su svi podaci i metapodaci organizovani kao fajlovi unutar jasno definisane hijerarhije direktorijuma, kopiranje tih fajlova u suštini i jeste backup, a njihovo vraćanje jeste restore. Međutim, činjenica da se radi o aktivnom sistemu koji kontinuirano upisuje nove podatke znači da naivno kopiranje u proizvoljnom trenutku i redosledu može rezultovati neusaglašenim snimkom, tj stanjem u kome su određene komponente novije od drugih, što pri restore-u može dovesti do nepotpunog ili neispravnog oporavka. Upravo zato obe procedure zahtevaju pažljivo poštovanje propisanog redosleda, što je tema narednih poglavlja.

5. Backup mehanizam

5.1. Preporučeni redosled backup-a

Kako bi se rizik od neusaglašenih snimaka sveo na minimum, zvanična dokumentacija propisuje konkretan redosled kojim se komponente kopiraju. Ovaj redosled nije proizvoljan već je zasnovan na međuzavisnostima između komponenti i na tome koja od njih se menja najređe, a koja najčešće.

Preporučeni redosled kopiranja je sledeći:

1. `<node_id>/snapshots/` - snapshot fajlovi koji definišu konzistentno stanje Parquet fajlova,
2. `<node_id>/dbs/` - Parquet fajlovi sa trajno sačuvanim vremensko-serijskim podacima,
3. `<node_id>/wal/` - WAL fajlovi sa podacima još uvek nesačuvani u Parquet formatu,
4. `<node_id>/catalog/` - log fajlovi koji prate promene kataloškog stanja,
5. `<node_id>/_catalog_checkpoint` - checkpoint fajl koji predstavlja sažeto trenutno stanje kataloga.

Logika ovog redosleda oslanja se na prirodu međuzavisnosti između komponenti. Snapshot fajlovi se kopiraju prvi jer definišu koje Parquet fajlove sistem smatra konzistentnim jer kopiranjem snapshot-a pre samih Parquet fajlova obezbeđuje se da backup ne sadrži Parquet fajlove na koje nijedan snapshot ne referiše, što bi ih činilo neupotrebljivim pri oporavku. Parquet fajlovi se kopiraju odmah nakon snapshot-ova jer predstavljaju najveći deo podataka i najređe se menjaju u poređenju sa WAL-om. WAL se kopira nakon Parquet fajlova jer sadrži najsvežije podatke koji se aktivno menjaju tokom rada sistema. Na kraju se kopiraju Catalog i checkpoint fajl, koji se kopiraju poslednji jer predstavljaju "pogled odozgo" na celokupno stanje sistema, kopiranjem Catalog-a na kraju smanjuje se verovatnoća da checkpoint opisuje stanje koje nije konzistentno sa već kopiranim Parquet fajlovima i WAL-om.

5.2. Kako izvoditi backup

Zvanična dokumentacija eksplicitno preporučuje da se backup izvodi tokom perioda zastoja sistema ili perioda minimalnog opterećenja. Razlog je jasan, fajlovi koji se kopiraju dok je sistem aktivan mogu se promeniti između početka i kraja kopiranja, što rezultuje nekonzistentnim snimkom.

Kada zastoj nije moguć, preporučuje se razmatranje alternativnih pristupa koji su dostupni na nivou infrastrukture, a ne na nivou InfluxDB-a samog. Ukoliko object storage backend to podržava, preporučuje se korišćenje mehanizama snapshot-ovanja na nivou fajl sistema ili storage sistema, koji mogu kreirati konzistentnu kopiju u jednom atomičnom koraku. Kompresija backup arhive nije obavezna, ali se preporučuje za dugoročno čuvanje radi uštede prostora.

5.3. Backup na lokalom fajl sistemu

Kada InfluxDB 3 Core koristi lokalni fajl sistem kao object storage backend, backup se izvodi kopiranjem direktorijuma u vremenski označenu backup lokaciju. Sledeći primer prikazuje bash skriptu koja implementira preporučeni redosled kopiranja:

```
#!/bin/bash
NODE_ID="NODE_ID"
DATA_DIR="/path/to/data"
BACKUP_DIR="/backup/$(date +%Y%m%d-%H%M%S)"

mkdir -p "$BACKUP_DIR"

# Copy in recommended order
cp -r $DATA_DIR/${NODE_ID}/snapshots "$BACKUP_DIR/"
cp -r $DATA_DIR/${NODE_ID}/dbs "$BACKUP_DIR/"
cp -r $DATA_DIR/${NODE_ID}/wal "$BACKUP_DIR/"
cp -r $DATA_DIR/${NODE_ID}/catalog "$BACKUP_DIR/"
cp $DATA_DIR/${NODE_ID}/_catalog_checkpoint "$BACKUP_DIR/"

echo "Backup completed to $BACKUP_DIR"
```

Promenljiva `NODE_ID` treba da odgovara vrednosti konfiguracionog parametra `--node-id` kojim je instanca pokrenuta. Promenljiva `DATA_DIR` treba da pokazuje na koren direktorijuma u kome InfluxDB 3 Core čuva podatke, što u slučaju Docker okruženja odgovara putanji koja je montirana kao volume.

Naziv backup direktorijuma generiše se automatski na osnovu trenutnog datuma i vremena u formatu `YYYYMMDD-HHMMSS`, što olakšava identifikaciju i upravljanje višestrukim backup-ovima tokom vremena.

5.4. Backup na S3-kompatibilnom skladištu

Kada InfluxDB 3 Core koristi AWS S3 ili S3-kompatibilno skladište kao object storage backend, backup se izvodi sinhronizacijom sadržaja između izvornog i odredišnog S3 bucket-a uz pomoć AWS CLI alata. Preporučeni redosled kopiranja ostaje identičan, a implementira se sledećom skriptom:

```
#!/bin/bash
NODE_ID="NODE_ID"
SOURCE_BUCKET="SOURCE_BUCKET"
BACKUP_BUCKET="BACKUP_BUCKET"
BACKUP_PREFIX="backup-$(date +%Y%m%d-%H%M%S)"

# Copy in recommended order
aws s3 sync s3://${SOURCE_BUCKET}/${NODE_ID}/snapshots \
  s3://${BACKUP_BUCKET}/${BACKUP_PREFIX}/${NODE_ID}/snapshots/

aws s3 sync s3://${SOURCE_BUCKET}/${NODE_ID}/dbs \
  s3://${BACKUP_BUCKET}/${BACKUP_PREFIX}/${NODE_ID}/dbs/

aws s3 sync s3://${SOURCE_BUCKET}/${NODE_ID}/wal \
  s3://${BACKUP_BUCKET}/${BACKUP_PREFIX}/${NODE_ID}/wal/
```

```
aws s3 sync s3://${SOURCE_BUCKET}/${NODE_ID}/catalog \
s3://${BACKUP_BUCKET}/${BACKUP_PREFIX}/${NODE_ID}/catalog/
```

```
aws s3 cp s3://${SOURCE_BUCKET}/${NODE_ID}/_catalog_checkpoint \
s3://${BACKUP_BUCKET}/${BACKUP_PREFIX}/${NODE_ID}/
```

```
echo "Backup completed to s3://${BACKUP_BUCKET}/${BACKUP_PREFIX}"
```

Pored `NODE_ID`, potrebno je zameniti `SOURCE_BUCKET` nazivom S3 bucket-a u kome InfluxDB 3 Core čuva podatke, i `BACKUP_BUCKET` nazivom odredišnog bucket-a u koji se backup upisuje. Korišćenje zasebnog bucket-a za backup preporučuje se radi fizičke odvojenosti podataka i radi mogućnosti primene zasebnih politika pristupa i čuvanja.

5.5. Šta backup ne obuhvata

Direktorijum `<node_id>/table-snapshots/` namerno se izostavlja iz backup procedure. Ovi fajlovi se periodično prepisuju tokom rada sistema i automatski se regenerišu pri svakom restartu instance, zbog čega njihovo odsustvo ne ugrožava ispravnost oporavka. Njihovo uključivanje u backup nije štetno, ali nepotrebno povećava veličinu backup arhive.

6. Restore mehanizam

6.1. Priroda restore operacije

Restore u InfluxDB 3 Core nije simetrična operacija u odnosu na backup, nije dovoljno jednostavno kopirati fajlove u suprotnom smeru i pokrenuti sistem. Restore podrazumeva vraćanje međusobno zavisnih komponenti u precizno određenom redosledu, pri čemu pogrešan redosled može rezultovati stanjem u kome sistem startuje, ali podaci nisu potpuni ili Catalog nije usaglašen sa Parquet fajlovima.

Pored redosleda, restore zahteva i ispunjenje nekoliko preduslova koji su obavezni bez izuzetka. Pre početka procedure neophodno je zaustaviti instancu InfluxDB 3 Core jer izvođenje restore-a nad aktivnim sistemom nije podržano i može dovesti do nepredvidivog ponašanja. Opciono, ali preporučeno, jeste brisanje postojećih podataka pre restore-a kako bi se osiguralo čisto početno stanje bez ostataka prethodnih fajlova koji bi mogli da interferiraju sa vraćenim podacima.

6.2. Preporučeni redosled restore-a

Redosled restore-a obrnut je u odnosu na redosled backup-a, ali ne mehanički, odnosno svaki korak ima konkretno obrazloženje zasnovano na međuzavisnostima komponenti.

Preporučeni redosled vraćanja komponenti je sledeći:

1. `<node_id>/_catalog_checkpoint` - checkpoint fajl kataloga,
2. `<node_id>/catalog/` - log fajlovi kataloga,
3. `<node_id>/wal/` - WAL fajlovi,
4. `<node_id>/dbs/` - Parquet fajlovi sa persistovanim podacima,
5. `<node_id>/snapshots/` - snapshot fajlovi.

Logika ovog redosleda zasniva se na tome kojim redosledom sistem koristi ove komponente pri pokretanju. Sistem pri startu najpre učitava Catalog checkpoint fajl kako bi rekonstruisao trenutno stanje metapodataka. Zbog toga on mora biti vraćen prvi, pre bilo kojih podatkovnih fajlova. Odmah nakon toga vraćaju se Catalog log fajlovi, koji dopunjuju sliku stanja metapodataka od poslednjeg checkpoint-a. WAL fajlovi se vraćaju pre Parquet fajlova jer sadrže podatke koji logički prethode Parquet persistenciji u toku upisa. Parquet fajlovi se vraćaju nakon WAL-a jer predstavljaju starije podatke. Snapshot fajlovi se vraćaju poslednji jer njihova uloga postaje relevantna tek kada su svi ostali fajlovi na svom mestu, snapshot definiše koji Parquet fajlovi se smatraju konzistentnim, pa mora biti vraćen nakon samih Parquet fajlova.

6.3. Restore na lokalnom fajl sistemu

Sledeći primer prikazuje bash skriptu koja implementira potpunu restore proceduru na lokalnom fajl sistemu, uključujući zaustavljanje servisa, opciono brisanje postojećih podataka, vraćanje komponenti u propisanom redosledu i podešavanje dozvola:

```
#!/bin/bash
NODE_ID="NODE_ID"
```

```
BACKUP_DIR="/backup/BACKUP_DATE"
DATA_DIR="/path/to/data"
```

1. Zaustavljanje InfluxDB-a

```
systemctl stop influxdb3 || docker stop influxdb3-core
```

2. Opciono: brisanje postojećih podataka radi čistog restore-a

```
rm -rf ${DATA_DIR}/${NODE_ID}/*
```

3. Vraćanje komponenti u propisanom redosledu

```
mkdir -p ${DATA_DIR}/${NODE_ID}
cp ${BACKUP_DIR}/_catalog_checkpoint ${DATA_DIR}/${NODE_ID}/
cp -r ${BACKUP_DIR}/catalog ${DATA_DIR}/${NODE_ID}/
cp -r ${BACKUP_DIR}/wal ${DATA_DIR}/${NODE_ID}/
cp -r ${BACKUP_DIR}/dbs ${DATA_DIR}/${NODE_ID}/
cp -r ${BACKUP_DIR}/snapshots ${DATA_DIR}/${NODE_ID}/
```

4. Podešavanje dozvola

```
chown -R influxdb:influxdb ${DATA_DIR}/${NODE_ID}
```

5. Pokretanje InfluxDB-a

```
systemctl start influxdb3 || docker start influxdb3-core
```

Promenljiva `BACKUP_DATE` treba da odgovara nazivu backup direktorijuma koji se želi obnoviti, u formatu koji je kreiran tokom backup procedure, na primer 20240115-143022. Korak podešavanja dozvola putem `chown` komande posebno je važan u Docker okruženjima, gde kontejner može da koristi drugačijeg sistemskog korisnika od onog koji je izvršio restore, što bi onemogućilo čitanje vraćenih fajlova.

6.4. Restore na S3-kompatibilnom skladištu

Kada InfluxDB 3 Core koristi S3 ili S3-kompatibilno skladište, restore se izvodi sinhronizacijom sadržaja iz backup bucket-a u ciljni bucket. Za razliku od lokalnog fajl sistema gde se komponente vraćaju jedna po jedna, S3 restore može da se izvede jednom `aws s3 sync` komandom budući da S3 operacije nisu podložne istim rizicima redosleda kao lokalno kopiranje fajlova u aktivnom sistemu, restore se ionako izvodi nad zaustavljenom instancom:

```
#!/bin/bash
NODE_ID="NODE_ID"
BACKUP_BUCKET="BACKUP_BUCKET"
BACKUP_PREFIX="backup-BACKUP_DATE"
TARGET_BUCKET="TARGET_BUCKET"
```

1. Zaustaviti InfluxDB

2. Opciono: brisanje postojećih podataka

```
aws s3 rm s3://${TARGET_BUCKET}/${NODE_ID} --recursive
```

3. Vraćanje iz backup-a

```
aws s3 sync s3://${BACKUP_BUCKET}/${BACKUP_PREFIX}/${NODE_ID}/\  
s3://${TARGET_BUCKET}/${NODE_ID}/
```

4. Pokrenuti InfluxDB

Pored `NODE_ID` i `BACKUP_DATE`, potrebno je zameniti `BACKUP_BUCKET` nazivom bucket-a koji sadrži backup i `TARGET_BUCKET` nazivom odredišnog bucket-a u koji se podaci vraćaju. Napomene o pokretanju i zaustavljanju instance ostavljene su kao komentari jer konkretna implementacija zavisi od načina na koji je instanca postavljena-sistemske servise, Docker kontejner ili druga forma orkestracije.

6.5. Očekivanja nakon restore-a i ograničenja oporavka

Uspešan restore ne garantuje nužno povratak svih podataka koji su ikada bili upisani u sistem. Oporavak je konzistentan do tačke u vremenu koja odgovara poslednjem snapshot-u uključenom u backup, ovo je fundamentalno ograničenje koje direktno proizlazi iz arhitekture sistema opisane u poglavlju 3.1.

Konkretno, podaci koji su upisani nakon što je poslednji snapshot kreiran, a čiji WAL fajlovi su u međuvremenu obrisani pre nego što je backup izveden, neće biti prisutni nakon restore-a. Pored toga, Parquet fajlovi koji postoje u backup-u, ali na koje nijedan snapshot fajl ne referiše, biće ignorisani pri oporavku, sistem ih neće uzeti u obzir čak i ako su fizički prisutni.

Ovo ograničenje ima praktičnu implikaciju na učestalost backup-ova: što se backup izvodi ređe, veći je potencijalni gubitak podataka u slučaju potrebe za oporavkom. S obzirom na to da InfluxDB 3 Core podrazumevano kreira nove Parquet snapshot-ove na svakih deset minuta, backup koji se izvodi jednom dnevno izlaže sistem potencijalnom gubitku do 24 sata podataka u najgorem slučaju, tačnije, svih podataka upisanih od poslednjeg snapshot-a do trenutka pada sistema.

6.6. Posebna razmatranja za Docker okruženje

Kada se InfluxDB 3 Core izvršava unutar Docker kontejnera, restore procedura zahteva dodatnu pažnju u pogledu konzistentnosti volume mount-ova i dozvola fajlova. Volume mount-ovi koji se koriste pri restore-u moraju biti identični onima koji su korišćeni pri normalnom radu instance, neslaganje putanja može uzrokovati da sistem ne pronađe vraćene fajlove čak i kada su oni fizički prisutni na disku. Dozvole fajlova moraju biti podešene tako da korisnik unutar kontejnera ima pravo čitanja nad svim vraćenim fajlovima, što se postiže *chown* komandom prikazanom u skripti u poglavlju 7.2.3. Preporučuje se i montiranje zasebnog backup direktorijuma kao dodatnog volume-a tokom izvođenja restore-a, kako bi se fajlovi mogli kopirati iz hosta u kontejner bez dodatnih koraka.

7. Praktična primena

Za potrebe demonstracije rada sa InfluxDB 3 Core bazom realizovan je IoT scenario u kome tri simulirana senzora periodično šalju merenja temperature i vlažnosti vazduha. Celokupno okruženje je kontejnerizovano pomoću Docker Compose alata, čime je obezbeđena ponovljivost postavke i jednostavno pokretanje svih komponenti jednom komandom.

Klijentska aplikacija (simulator) implementirana je u Node.js okruženju, pri čemu je korišćena zvanična *influxdata/influxdb3-client* biblioteka za komunikaciju sa bazom.

Korišćene su dve Docker slike:

- **influxdb:3-core** - sama baza podataka. Servis je u kompoziciji nazvan *influxdb3* i izložen na portu **8181**, koji predstavlja HTTP API tačku za upis i izvršavanje upita.
- **influxdata/influxdb3-ui:1.7.0** - grafički interfejs (*InfluxDB 3 Explorer*) korišćen za interaktivni rad sa bazom, izvršavanje SQL upita i analizu rezultata. Servis se pokreće sa parametrom **--mode=admin** i izložen je na portu 8888.

```
influxdb3:
  image: influxdb:3-core
  container_name: influxdb3-core
  restart: unless-stopped
  ports:
    - "8181:8181"
  command:
    - influxdb3
    - serve
    - --node-id=node0
    - --object-store=file
    - --data-dir=/var/lib/influxdb3
    - --wal-flush-interval=5s
    - --wal-snapshot-size=10
  volumes:
    - influxdb3-data:/var/lib/influxdb3
```

Slika 7– influxdb:3-core slika

```
influxdb3-explorer:
  image: influxdata/influxdb3-ui:1.7.0
  container_name: influxdb3-explorer
  restart: unless-stopped
  depends_on:
    - influxdb3
  ports:
    - "8888:8080"
  command:
    - --mode=admin
```

Slika 8 - influxdata/influxdb3-ui:1.7.0

Trajno čuvanje podataka realizovano je korišćenjem imenovanog Docker volumena *influxdb3-data*, koji je u kontejneru montiran na putanju */var/lib/influxdb3*. Na taj način Parquet fajlovi i WAL segmenti zadržavaju se i nakon zaustavljanja kontejnera, budući da su fizički smešteni van kontejnerskog fajl sistema.

Kao što je već opisano u poglavlju 3.2, podaci se pri upisu najpre zapisuju u Write-Ahead Log fajlove pre konverzije u Parquet format. Učestalost zatvaranja jednog WAL segmenta i učestalost kreiranja Parquet snapshot-a ključno utiču na to koliko brzo se sveži podaci pojavljuju u sistemskoj tabeli *system.parquet_files* i koliko su potencijalni gubici podataka u slučaju otkaza. U konfiguraciji servisa korišćeni su sledeći parametri:

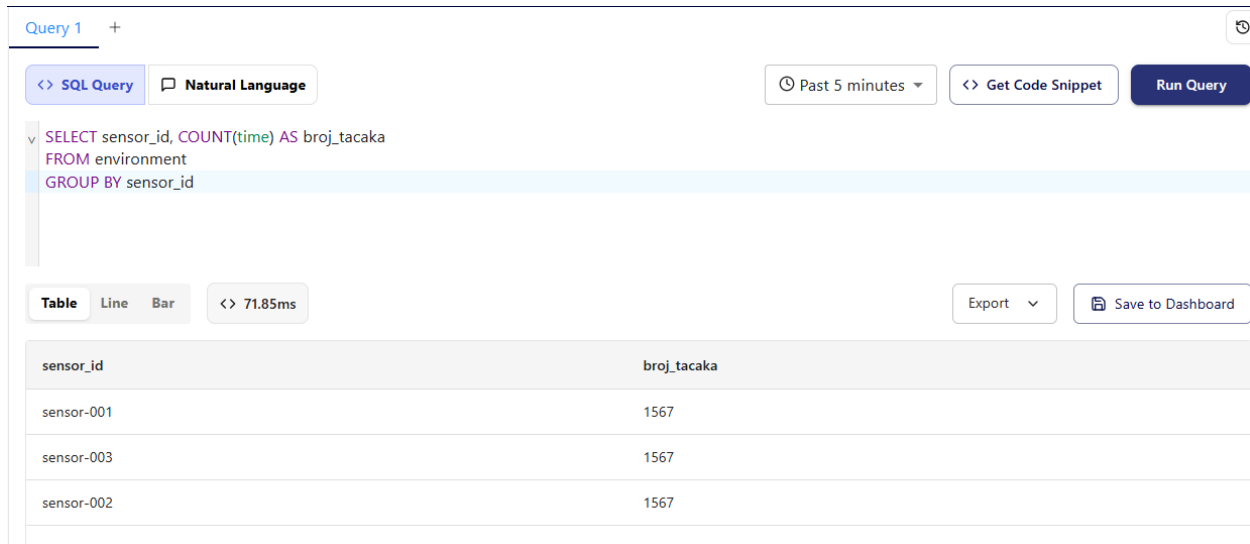
- **--wal-flush-interval=5s** - trajanje WAL segmenta postavljeno je na 5 sekundi. To znači da se aktivni segment zatvara i zapisuje u object storage svakih pet sekundi, čime se ograničava potencijalni gubitak podataka u slučaju otkaza i obezbeđuje predvidiv ritam generisanja novih segmenata.

- **--wal-snapshot-size=10** - broj WAL segmenata nakon kojeg se kreira novi Parquet snapshot postavljen je na 10. U kombinaciji sa **--wal-flush-interval=5s**, snapshot se formira na svakih $10 \times 5s = 50$ sekundi, što znači da Parquet fajlovi postaju vidljivi u sistemskoj tabeli *system.parquet_files* već nakon manje od jednog minuta rada simulatora. Radi poređenja, podrazumevana vrednost ovog parametra iznosi 600 segmenata, sa podrazumevanim flush intervalom od 1 sekunde to odgovara snapshot-u na svakih 10 minuta, dok bi sa flush intervalom od 5 sekundi isti snapshot size rezultovao čekanjem od približno 50 minuta. Ova izmena je od ključnog značaja za demonstracioni scenario, jer omogućava posmatranje efekata backup i restore procedure u realnom vremenu, bez potrebe za dugotrajnim čekanjem na kreiranje Parquet fajlova.

7.1. Backup

7.1.1. Priprema i verifikacija stanja pre backup-a

Pre pokretanja backup procedure zabeleženo je referentno stanje baze. U tu svrhu izvršen je SQL upit koji prebrojava tačke po senzoru:



Query 1 +

<> SQL Query Natural Language

Past 5 minutes <> Get Code Snippet Run Query

```
SELECT sensor_id, COUNT(time) AS broj_tacaka
FROM environment
GROUP BY sensor_id
```

Table Line Bar <> 71.85ms Export Save to Dashboard

sensor_id	broj_tacaka
sensor-001	1567
sensor-003	1567
sensor-002	1567

Slika 9 - Broj tačaka po senzoru pre backup-a

Rezultat upita pokazuje da su sva tri senzora upisala jednak broj merenja: svaki po 1567 tačaka, što daje ukupno 4701 merenje u tabeli *environment*.

7.1.2. Izvođenje backup procedure

Backup je realizovan PowerShell skriptom *backup-influxdb.ps1* koja implementira redosled kopiranja propisan zvaničnom dokumentacijom.

```

$Container = "influxdb3-core"
$NodeId = "node0"
$DataDir = "/var/lib/influxdb3"
$BackupRoot = ".\backups"

$timestamp = Get-Date -Format "yyyyMMdd-HH:mm:ss"
$BackupDir = Join-Path $BackupRoot "backup-$timestamp"
New-Item -ItemType Directory -Path $BackupDir -Force | Out-Null

Write-Host ""
Write-Host " InfluxDB 3 Core - Backup"
Write-Host " Kontejner : $Container"
Write-Host " Node ID : $NodeId"
Write-Host " Odrediste : $BackupDir"
Write-Host ""

```

Slika 10 — Parametri i inicijalizacija backup skripte backup-influxdb.ps1

```

Write-Host "[1/3] Stop InfluxDB"
docker stop $Container
Write-Host ""

Write-Host "[2/3] Copy Data"

$SnapDir = Join-Path $BackupDir "snapshots"
$DbsDir = Join-Path $BackupDir "dbs"
$WalDir = Join-Path $BackupDir "wal"
$CatalogDir = Join-Path $BackupDir "catalog"
$CheckpointDir = Join-Path $BackupDir "_catalog_checkpoint"

docker cp "${Container}:${DataDir}/${NodeId}/snapshots" "$SnapDir"
docker cp "${Container}:${DataDir}/${NodeId}/dbs" "$DbsDir"
docker cp "${Container}:${DataDir}/${NodeId}/wal" "$WalDir"
docker cp "${Container}:${DataDir}/${NodeId}/catalog" "$CatalogDir"
docker cp "${Container}:${DataDir}/${NodeId}/db-indices" "$BackupDir\db-indices"
docker cp "${Container}:${DataDir}/${NodeId}/snapshot-checkpoints" "$BackupDir\snapshot-checkpoints"

Write-Host ""

Write-Host "[3/3] Start InfluxDB"
docker start $Container
Write-Host ""

```

Slika 11 — Implementacija backup koraka u skripti backup-influxdb.ps1

Skripta izvršava tri koraka. U prvom koraku zaustavlja se kontejner influxdb3-core. Zaustavljanje kontejnera pre kopiranja fajlova obavezno je jer kopiranje aktivne baze može rezultovati nekonzistentnim backup-om.

U drugom koraku kopiraju se sve komponente u propisanom redosledu korišćenjem *docker cp* komande, koja radi i nad zaustavljenim kontejnerima.

U trećem koraku kontejner se ponovo pokreće kako bi sistem nastavio sa normalnim radom.

Skripta se pokreće sledećom komandom:

```
powershell -ExecutionPolicy Bypass -File .\scripts\backup-influxdb.ps1
```

```
PS powershell -ExecutionPolicy Bypass -File .\scripts\backup-influxdb.ps1

InfluxDB 3 Core - Backup
Kontejner : influxdb3-core
Node ID   : node0
Određiste : .\backups\backup-20260524-151318

[1/3] Stop InfluxDB
influxdb3-core

[2/3] Copy Data
Successfully copied 111kB to \backups\backup-20260524-151318\snapshots
Successfully copied 403kB to \backups\backup-20260524-151318\dfs
Successfully copied 323kB to \backups\backup-20260524-151318\wal
Successfully copied 36.4kB to \backups\backup-20260524-151318\catalog
Successfully copied 23.6kB to \backups\backup-20260524-151318\db-indices
Successfully copied 24.1kB to \backups\backup-20260524-151318\snapshot-checkpoints

[3/3] Start InfluxDB
influxdb3-core

Backup završen: .\backups\backup-20260524-151318
```

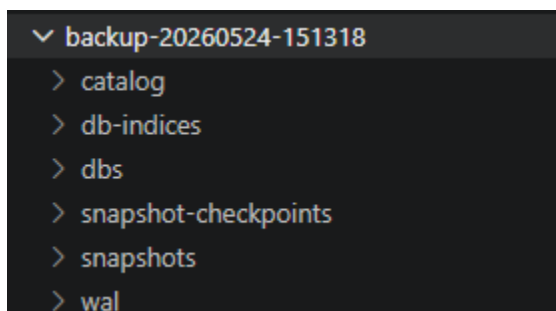
Slika 12 - Ispis u terminalu po završetku procedure

Naziv backup direktorijuma generiše se automatski na osnovu trenutnog datuma i vremena u formatu YYYYMMDD-HHMMSS, što omogućava jednoznačnu identifikaciju svakog backup-a i jednostavno upravljanje višestrukim backup-ovima tokom vremena.

Važno je napomenuti da stvarna struktura direktorijuma u kontejneru odstupa od one opisane u zvaničnoj dokumentaciji. Umesto fajla `_catalog_checkpoint`, u korišćenoj verziji InfluxDB 3 Core postoji direktorijum `snapshot-checkpoints`, a prisutan je i dodatni direktorijum `db-indices` koji dokumentacija ne pominje. Oba direktorijuma su uključena u backup proceduru. Direktorijum `table-snapshots`, koji dokumentacija eksplicitno navodi kao nepotraban za backup, u ovoj verziji nije prisutan, što je konzistentno sa napomenom da se regeneriše pri pokretanju servera.

7.1.3. Verifikacija sadržaja backup-a

Po završetku procedure, backup direktorijum `backup-20260524-151318` sadrži sledeće komponente:



Slika 13 – Verifikacija dostupnosti podataka nakon backup-a

Prisustvo svih šest komponenti: catalog, db-indices, dbs, snapshot-checkpoints, snapshots i wal potvrđuje da je backup obuhvatio celokupno stanje instance, uključujući i sačuvane Parquet fajlove i WAL segmente sa podacima koji još uvek nisu konvertovani u Parquet format.

Nakon ponovnog pokretanja kontejnera u koraku [3/3], sistem je nastavio sa normalnim radom. Verifikacioni upit potvrđuje da su podaci dostupni i da je broj tačaka po senzoru nepromenjen: sva tri senzora i dalje pokazuju po 1567 tačaka, što znači da zaustavljanje kontejnera tokom backup procedure nije uzrokovalo gubitak podataka.

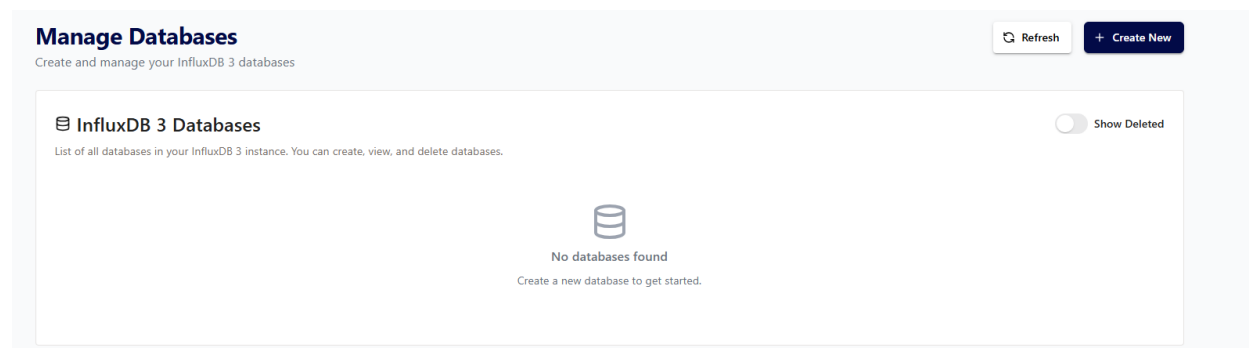
7.2. Restore

7.2.1. Simulacija gubitka podataka

Pre izvođenja restore procedure neophodno je simulirati scenario gubitka podataka. U tu svrhu izvršeno je brisanje celokupnog sadržaja *node0* direktorijuma iz Docker volumena.

Budući da *docker exec* ne radi nad zaustavljenim kontejnerima, brisanje je izvedeno kroz Alpine helper kontejner koji montira isti volumen:

1. `docker stop influxdb3-core`
2. `docker run --rm -v "project_influxdb3-data:/data" alpine:3.20 sh -c "rm -rf /data/node0/*"`
3. `docker start influxdb3-core`



Slika 14 — Stanje sistema nakon brisanja podataka: "No databases found"

Nakon ponovnog pokretanja kontejnera, Explorer prikazuje poruku "*No databases found*": Catalog je obrisao i sistem nema nikakvih informacija o prethodno postojećim bazama i tabelama, što potvrđuje da su podaci u potpunosti izgubljeni.

7.2.2. Izvođenje restore procedure

Restore je realizovan PowerShell skriptom `restore-influxdb.ps1`.


```

Write-Host "[1/5] Stop InfluxDB"
docker stop $Container
Write-Host ""

Write-Host "[2/5] Brisanje postojećih podataka..."
docker run --rm `
    -v "${Volume}:/data" `
    alpine:3.20 `
    sh -c "rm -rf /data/$NodeId && mkdir -p /data/$NodeId"
Write-Host ""

Write-Host "[3/5] Restore Data"
docker cp "$BackupDir\snapshot-checkpoints" "${Container}:${DataDir}/${NodeId}/"
docker cp "$BackupDir\catalog" "${Container}:${DataDir}/${NodeId}/"
docker cp "$BackupDir\wal" "${Container}:${DataDir}/${NodeId}/"
docker cp "$BackupDir\dbs" "${Container}:${DataDir}/${NodeId}/"
docker cp "$BackupDir\snapshots" "${Container}:${DataDir}/${NodeId}/"
docker cp "$BackupDir\db-indices" "${Container}:${DataDir}/${NodeId}/"
Write-Host ""

Write-Host "[4/5] Podesavanje dozvola (chown 1500:1500)..."
docker run --rm `
    -v "${Volume}:/data" `
    alpine:3.20 `
    sh -c "chown -R 1500:1500 /data/$NodeId"
Write-Host ""

Write-Host "[5/5] Start InfluxDB"
docker start $Container
Write-Host ""

```

Slika 15 — Implementacija restore koraka u skripti restore-influxdb.ps1

Skripta izvršava pet koraka. U prvom koraku zaustavlja se kontejner influxdb3-core jer restore nad aktivnim sistemom nije podržan.

U drugom koraku briše se eventualni preostali sadržaj *node0* direktorijuma kroz Alpine helper kontejner i kreira se prazan direktorijum spreman za prijem podataka.

U trećem koraku komponente se vraćaju u propisanom redosledu, obrnutom od redosleda backup-a, korišćenjem *docker cp* komande.

U četvrtom koraku podešavaju se dozvole fajlova kroz Alpine helper kontejner, razlog za ovaj korak detaljno je opisan u poglavlju 7.2.3.

U petom koraku kontejner se ponovo pokreće.

Skripta se pokreće sledećom komandom:

powershell -ExecutionPolicy Bypass -File .\scripts\restore-influxdb.ps1

```
powershell -ExecutionPolicy Bypass -File .\scripts\restore-influxdb.ps1 -BackupDir .\backups\backup-20260524-151318

InfluxDB 3 Core - Restore
Kontejner : influxdb3-core
Node ID   : node0
Izvor     : .\backups\backup-20260524-151318

[1/4] Stop InfluxDB
influxdb3-core

[2/4] Brisanje postojećih podataka...

[3/4] Restore Data
Successfully copied 24.1kB to influxdb3-core:/var/lib/influxdb3/node0/
Successfully copied 36.4kB to influxdb3-core:/var/lib/influxdb3/node0/
Successfully copied 323kB to influxdb3-core:/var/lib/influxdb3/node0/
Successfully copied 403kB to influxdb3-core:/var/lib/influxdb3/node0/
Successfully copied 111kB to influxdb3-core:/var/lib/influxdb3/node0/
Successfully copied 23.6kB to influxdb3-core:/var/lib/influxdb3/node0/

[4/4] Start InfluxDB
influxdb3-core

Restore završen.
```

Slika 16 — Ispis terminala po završetku restore procedure

7.2.3. Problem sa dozvolama i njegovo rešavanje

Nakon prvog pokušaja restore-a, Docker logovi su pokazali grešku koja je sprečavala normalan rad sistema:

```
2026-05-24T13:42:27.084596Z WARN object_store_utils::retryable_object_store: Persisting table index snapshot for table 0 at sequence 9: Retrying object store put operation for node0/table-snapshots/1/0/000000
000000000000009.info.json after error: Generic LocalFileSystem error: Unable to create dir /var/lib/influxdb3/node0/table-snapshots/1/0: Permission denied (os error 13). Retry after 2244ms
2026-05-24T13:42:27.194353Z WARN object_store_utils::retryable_object_store: Persisting table index snapshot for table 0 at sequence 7: Retrying object store put operation for node0/table-snapshots/1/0/000000
000000000000007.info.json after error: Generic LocalFileSystem error: Unable to create dir /var/lib/influxdb3/node0/table-snapshots/1/0: Permission denied (os error 13). Retry after 2897ms
2026-05-24T13:42:27.374891Z WARN object_store_utils::retryable_object_store: Persisting table index snapshot for table 0 at sequence 16: Retrying object store put operation for node0/table-snapshots/1/0/000000
000000000000016.info.json after error: Generic LocalFileSystem error: Unable to create dir /var/lib/influxdb3/node0/table-snapshots/1/0: Permission denied (os error 13). Retry after 1807ms
2026-05-24T13:42:27.389284Z WARN object_store_utils::retryable_object_store: Persisting table index snapshot for table 0 at sequence 20: Retrying object store put operation for node0/table-snapshots/1/0/000000
000000000000020.info.json after error: Generic LocalFileSystem error: Unable to create dir /var/lib/influxdb3/node0/table-snapshots/1/0: Permission denied (os error 13). Retry after 1707ms
2026-05-24T13:42:27.409281Z WARN object_store_utils::retryable_object_store: Persisting table index snapshot for table 0 at sequence 12: Retrying object store put operation for node0/table-snapshots/1/0/000000
000000000000012.info.json after error: Generic LocalFileSystem error: Unable to create dir /var/lib/influxdb3/node0/table-snapshots/1/0: Permission denied (os error 13). Retry after 2294ms
2026-05-24T13:42:27.454661Z WARN object_store_utils::retryable_object_store: Persisting table index snapshot for table 0 at sequence 13: Retrying object store put operation for node0/table-snapshots/1/0/000000
000000000000013.info.json after error: Generic LocalFileSystem error: Unable to create dir /var/lib/influxdb3/node0/table-snapshots/1/0: Permission denied (os error 13). Retry after 2285ms
2026-05-24T13:42:27.495788Z WARN object_store_utils::retryable_object_store: Persisting table index snapshot for table 0 at sequence 15: Retrying object store put operation for node0/table-snapshots/1/0/000000
000000000000015.info.json after error: Generic LocalFileSystem error: Unable to create dir /var/lib/influxdb3/node0/table-snapshots/1/0: Permission denied (os error 13). Retry after 3181ms
2026-05-24T13:42:27.547853Z WARN object_store_utils::retryable_object_store: Persisting table index snapshot for table 0 at sequence 8: Retrying object store put operation for node0/table-snapshots/1/0/000000
000000000000008.info.json after error: Generic LocalFileSystem error: Unable to create dir /var/lib/influxdb3/node0/table-snapshots/1/0: Permission denied (os error 13). Retry after 2641ms
```

Slika 17 — Docker logovi sa greškom Permission denied pre ispravke dozvola

Uzrok greške bio je u tome što je Alpine helper kontejner kreirao *node0* direktorijum sa root dozvolama (uid=0), dok InfluxDB 3 Core unutar kontejnera radi kao korisnik influxdb3 sa uid=1500. Neslaganje dozvola onemogućilo je sistemu da kreira table-snapshots direktorijum koji se regeneriše pri svakom pokretanju.

UID korisnika unutar kontejnera prethodno je utvrđen komandom:

docker exec influxdb3-core id

```
uid=1500(influxdb3) gid=1500(influxdb3) groups=1500(influxdb3)
```

Slika 18 — Utvrđivanje UID-a korisnika unutar kontejnera

Greška je otklonjena izvršavanjem **chown** komande kroz Alpine helper kontejner sa ispravnim UID-om:

1. `docker stop influxdb3-core`
2. `docker run --rm -v "project_influxdb3-data:/data" alpine:3.20 sh -c "chown -R 1500:1500 /data/node0"`

3. docker start influxdb3-core

Ovaj korak je dodat u finalnu verziju skripte kao obavezan korak [4/5]. Ova situacija praktično potvrđuje napomenu iz poglavlja 6.6., podešavanje dozvola fajlova u Docker okruženju je neophodan i kritičan korak restore procedure koji se ne sme izostaviti.

```
InfluxDB 3 Core - Restore
Kontejner : influxdb3-core
Node ID   : node0
Izvor      : .\backups\backup-20260524-151318

[1/5] Stop InfluxDB
influxdb3-core

[2/5] Brisanje postojećih podataka...

[3/5] Restore Data
Successfully copied 24.1kB to influxdb3-core:/var/lib/influxdb3/node0/
Successfully copied 36.4kB to influxdb3-core:/var/lib/influxdb3/node0/
Successfully copied 323kB to influxdb3-core:/var/lib/influxdb3/node0/
Successfully copied 403kB to influxdb3-core:/var/lib/influxdb3/node0/
Successfully copied 111kB to influxdb3-core:/var/lib/influxdb3/node0/
Successfully copied 23.6kB to influxdb3-core:/var/lib/influxdb3/node0/

[4/5] Podešavanje dozvola (chown 1500:1500)...

[5/5] Start InfluxDB
influxdb3-core

Restore završen.
```

Slika 19 — Ispis terminala po završetku restore procedure

7.2.4. Verifikacija uspešnosti restore-a

Po pokretanju kontejnera nakon ispravke dozvola, Docker logovi potvrđuju uspešan oporavak sistema. Ključne poruke u logovima su:

- *replaying WAL files* - sistem čita WAL fajlove iz backup-a i rekonstruiše stanje podataka koji još nisu bili persistovani u Parquet format;
- *completed replaying wal files time_taken=1.67617ms* - WAL replay završen uspešno;
- *startup time: 1115ms address=0.0.0.0:8181* - server uspešno pokrenut i spreman za upite.

```
2026-05-24T13:46:20.394987Z INFO influxdb3_wal::object_store: replaying WAL files
2026-05-24T13:46:20.395674Z INFO influxdb3_wal::object_store: replaying WAL file with details n_ops=1 min_timestamp_ns=1778093402766000000 max_timestamp_ns=1778093402766000000 wal_file_number=1555 snapshot_details=Some(SnapshotDetails { snapshot_sequence_number: SnapshotSequenceNumber(69), end_time_marker: 1778093408000000000, first_wal_sequence_number: WalFileSequenceNumber(1533), last_wal_sequence_number: WalFileSequenceNumber(1554), forced: false })
2026-05-24T13:46:20.396668Z INFO influxdb3_wal::object_store: replaying WAL file with details n_ops=1 min_timestamp_ns=1778093417780000000 max_timestamp_ns=1778093417780000000 wal_file_number=1556 snapshot_details=None
```

Slika 20 - Docker logovi nakon uspešnog restore-a

```

2026-05-24T13:46:20.396662Z INFO influxdb3_wal::object_store: completed replaying wal files time_taken=1.67617ms
2026-05-24T13:46:20.396688Z INFO influxdb3_lib::commands::serve: setting up background mem check for query buffer
2026-05-24T13:46:20.396693Z INFO influxdb3_lib::commands::serve: setting up telemetry store
2026-05-24T13:46:20.396947Z INFO influxdb3_write::deleter: Started catalog hard deleter task. delete_grace_period=86400s
2026-05-24T13:46:21.467446Z INFO influxdb3_lib::commands::serve: setting up server with authz disabled for paths paths_without_authz=[]
2026-05-24T13:46:21.467598Z INFO influxdb3_server: startup time: 1115ms address=0.0.0.0:8181

```

Slika 21 - Docker logovi nakon uspešnog restore-a

Konačna verifikacija izvršena je istim SQL upitom koji je korišćen pre backup-a:

The screenshot shows the InfluxDB web interface. On the left, the 'Schema' panel displays the 'environment' table. The main area shows a SQL query: `SELECT sensor_id, COUNT(time) AS broj_tacaka FROM environment GROUP BY sensor_id`. Below the query, the results are displayed in a table format. The table has two columns: 'sensor_id' and 'broj_tacaka'. The results show three rows, each with a sensor ID and a count of 1567.

sensor_id	broj_tacaka
sensor-002	1567
sensor-001	1567
sensor-003	1567

Slika 22 - Verifikacija podataka nakon restore-a

Rezultat potvrđuje potpun oporavak: sva tri senzora pokazuju tačno po 1567 tačaka, što je identično stanju zabeleženom pre brisanja podataka. Ukupno 4701 merenje uspešno je vraćeno iz backup-a, bez ijednog izgubljenog zapisa.

5. Zaključak

Ovaj rad je kroz teorijsku analizu i praktičnu implementaciju prikazao backup i restore mehanizme u InfluxDB 3 Core. Arhitektura zasnovana na FDAP steku i object storage-u fundamentalno menja prirodu backup operacije, budući da sistem sve podatke i metapodatke čuva kao fajlove unutar jasno definisane hijerarhije direktorijuma, backup se u osnovi svodi na kopiranje tog skupa fajlova u rezervno odredište. Međutim, višekomponentna priroda stanja instance znači da naivno kopiranje bez poštovanja propisanog redosleda može rezultovati neusaglašenim snimkom iz kojeg oporavak nije moguć ili je nepotpun.

Ključna karakteristika sistema jeste odsustvo ugrađenih backup alata, odgovornost za ispravnost procedure u potpunosti leži na administratoru, što zahteva dobro razumevanje interne arhitekture i međuzavisnosti između komponenti.

Praktična implementacija potvrdila je teorijske postavke, ali je istovremeno otkrila odstupanja između dokumentacije i stvarnog stanja sistema, fajl *_catalog_checkpoint* nije prisutan u korišćenoj verziji, dok postoje dodatne komponente (*snapshot-checkpoints*, *db-indices*) koje dokumentacija ne pominje. Ovo ukazuje na to da je pregled stvarne strukture direktorijuma pre izvođenja backup procedure neophodan korak u svakoj produkcijskoj primeni. Pored toga, Docker okruženje zahteva eksplicitan korak podešavanja dozvola fajlova putem *chown* komande, bez njega sistem se pokreće, ali ne može da funkcioniše ispravno. Uspešnost procedure potvrđena je verifikacionim upitom koji je pokazao identičan broj merenja pre i nakon restore-a, ukupno 4701 tačka bez ijednog izgubljenog zapisa.

6. Literatura

- [1] „Backup and restore data” [Online]. Available: <https://docs.influxdata.com/influxdb3/core/admin/backup-restore/#backup-process>
- [2] „InfluxDB 3 storage engine architecture” [Online]. Available: <https://docs.influxdata.com/influxdb3/cloud-dedicated/reference/internals/storage-engine/>
- [3] „InfluxDB 3: System Architecture” [Online]. Available: <https://www.influxdata.com/blog/influxdb-3-0-system-architecture/>
- [4] „Flight, DataFusion, Arrow, and Parquet: Using the FDAP Architecture to build InfluxDB 3” [Online]. Available: <https://www.influxdata.com/blog/flight-datafusion-arrow-parquet-fdap-architecture-influxdb/>
- [5] „Interna struktura i organizacija indeksa kod influxdb baze podataka” [Online]. Available: <https://github.com/anitagolubovic/InfluxDB>